

TEMA 9: STL

Estructuras de Datos y Contenedores

1. Contenedores Secuenciales

a. vector

- **Descripción:** arreglo dinámico que permite acceso rápido a elementos mediante índice.
- **Funciones principales:**
- `push_back(elemento)` – Añade al final.
- `pop_back()` – Elimina el último elemento.
- `insert(pos, elemento)` – Inserta en posición.
- `erase(pos)` – Elimina elemento en posición.
- `size()` – Número de elementos.
- `capacity()` – Capacidad actual.
- `resize(n)` – Ajusta tamaño.
- `operator[]` o `at()` – Acceso a elementos.
- **Declaración:**

```
#include <vector>
std::vector<T> v;
```

b. list

- **Descripción:** lista doblemente enlazada.
- **Funciones principales:**
- `push_back(elemento)` – Inserta al final.
- `push_front(elemento)` – Inserta al principio.
- `pop_back()` / `pop_front()`.
- `insert(iterador, elemento)`.
- `erase(iterador)`.
- `size()`.
- **Declaración:**

```
#include <list>
std::list<T> lst;
```

c. deque

- **Descripción:** cola doble con acceso al azar.
- **Funciones principales:** similares a vector y lista.
- `push_back()`, `push_front()`, `pop_back()`, `pop_front()`, `insert()`, `erase()`.
- **Declaración:**

```
#include <deque>
std::deque<T> dq;
```

d. queue (adaptador)

- **Descripción:** cola FIFO, basada en deque.
- **Funciones principales:**
- `push()` – Inserta al final.
- `pop()` – Elimina del frente.
- `front()` – Elemento en frente.
- `back()` – Último elemento.
- **Declaración:**

```
#include <queue>
std::queue<T> q;
```

e. priority_queue

- **Descripción:** cola ordenada, siempre en orden descendente (máximo en frente).
- **Funciones principales:**
- `push()`, `pop()`, `top()`.
- **Declaración:**

```
#include <queue>
std::priority_queue<T> pq;
```

f. stack

- **Descripción:** estructura LIFO.
- **Funciones principales:**
- `push()`, `pop()`, `top()`, `size()`.
- **Declaración:**

```
#include <stack>
std::stack<T> s;
```

2. Contenedores Asociativos

a. map y multimap

- **Descripción:** mapeo clave-valor ordenado, con clave única (map) o múltiple (multimap).
- **Funciones principales:**
- `insert()`, `find()`, `erase()`, `operator[]`.
- **Declaración:**

```
#include <map>
std::map<K, V> m;
std::multimap<K, V> mm;
```

b. unordered_map y unordered_multimap

- **Descripción:** similares a map, pero sin orden en las claves, usando hashing.
- **Declaración:**

```
#include <unordered_map>
std::unordered_map<K, V> um;
std::unordered_multimap<K, V> umm;
```

c. set y multiset

- **Descripción:** colección ordenada de elementos únicos (set) o múltiple (multiset).
- **Funciones:** `insert()`, `find()`, `erase()`.
- **Declaración:**

```
#include <set>
std::set<T> s;
std::multiset<T> ms;
```

d. unordered_set y unordered_multiset

- **Descripción:** colección sin orden, basada en hashing.
- **Declaración:**

```
#include <unordered_set>
std::unordered_set<T> us;
std::unordered_multiset<T> ums;
```

Iteradores

Los **iteradores** en C++ son objetos que actúan como punteros abstractos, permitiendo recorrer, acceder y modificar elementos en un contenedor de forma uniforme y segura. Funcionan como enlaces entre los algoritmos y las estructuras de datos, facilitando la manipulación de colecciones como **vector**, **list**, **map**, entre otros.

Funcionamiento y sintaxis básica

Para recorrer un contenedor utilizando un iterador, típicamente se emplea una estructura de ciclo:

```
for (auto it = c.begin(); it != c.end(); ++it) {
    // Operaciones con *it
}
```

Tipos de iteradores y sus características

Tipo de iterador	Funciones principales	Operaciones permitidas	Uso típico
iterator	Desde el principio hasta el final del contenedor	Incrementar (++it) y desreferenciar (*it)	Recorrer y modificar elementos en contenedores como vector , list
const_iterator	Igual que iterator , pero sólo lectura (const)	Solo lectura (*it no modificable)	Acceso a datos sin modificar
reverse_iterator	Desde el final hacia el principio (rbegin() a rend())	Decrementar (--it) y desreferenciar (*it)	Recorrer contenedores en orden inverso

Propiedades y métodos principales

Método / Operador	Función	Ejemplo de uso
-------------------	---------	----------------

<code>operator*()</code>	Desreferenciar: obtener referencia al elemento	<code>*it</code>
<code>operator++()</code>	Avanzar al próximo elemento	<code>++it</code>
<code>operator--()</code>	Retroceder al elemento anterior (en iteradores bidireccionales)	<code>--it</code>
<code>operator==(())</code> y <code>operator!=(())</code>	Comparar dos iteradores	<code>it == c.end(), it != c.begin()</code>
<code>-></code>	Acceder a miembros de objetos apuntados	<code>it->miembro</code>

Operaciones adicionales útiles

- `c.begin(), c.end()` — inicio y fin del recorrido.
- `c.rbegin(), c.rend()` — recorrido en orden inverso.
- `distance(it1, it2)` — calcular distancia entre iteradores.
- `advance(it, n)` — avanzar o retroceder un iterador `n` pasos.

Ejemplos prácticos

Recorrer un vector e imprimir elementos

```
#include <vector>
#include <iostream>

int main() {
    std::vector<int> v = {1, 2, 3, 4, 5};
    for (auto it = v.begin(); it != v.end(); ++it) {
        std::cout << *it << " ";
    }
    std::cout << std::endl;
}
```

Recorrer en orden inverso usando `reverse_iterator`

```
#include <deque>
#include <iostream>

int main() {
    std::deque<int> d = {1, 2, 3, 4, 5};
    for (auto rit = d.rbegin(); rit != d.rend(); ++rit) {
        std::cout << *rit << " ";
    }
    std::cout << std::endl;
}
```

Uso con **const_iterator** para acceso en modo lectura

```
for (auto it = v.cbegin(); it != v.cend(); ++it) {
    std::cout << *it << " ";
}
```

Algoritmos

- Uso conjunto con iteradores, ejemplo:

```
#include <algorithm>
#include <vector>

std::vector<int> v = {1,2,3,4,5};
std::copy(v.begin(), v.end(),
std::ostream_iterator<int>(std::cout, " "));
std::sort(v.begin(), v.end());
auto it = std::find(v.begin(), v.end(), 3);
```

- Funciones comunes:
- **copy(), find(), remove(), sort(), reverse(), min_element(), max_element(), count(), accumulate(), for_each(), transform(), equal().**

Manejo de Archivos

1. Archivos de Texto

Apertura:

```
#include <fstream>
#include <string>

std::ifstream fin("archivo.txt");           // Abrir para lectura
while (fin.is_open()) {
    // Lectura
}

std::ofstream fout("salida.txt");           // Abrir para escritura
while (fout.is_open()) {
    // Escritura
}

std::ofstream fout("archivo.txt", std::ios::trunc); //Abrir en modo
truncamiento (borra el contenido previo)
```

*Nota: **trunc** es la opción por defecto al abrir para escritura en **ofstream**, así que generalmente no es necesario especificarlo explícitamente, pero puede hacerse para mayor claridad.*

```
std::ofstream outFile("archivo.txt", std::ios::app); //Modo eppend

// También en modo lectura y escritura:
std::fstream file("nombre.txt", std::ios::in | std::ios::out);

//Entrada y salida con trunc
std::fstream file("archivo.txt", std::ios::in | std::ios::out |
std::ios::trunc);

//Entrada y salida con append
std::fstream file("archivo.txt", std::ios::in | std::ios::out |
std::ios::app);
```

Uso con iteradores:

```
#include <iterator>
#include <algorithm>
#include <vector>

std::vector<int> v;
std::copy(std::istream_iterator<int>(fin),
std::istream_iterator<int>(),
std::back_inserter(v));
```

2. Archivos Binarios

Apertura:

```
#include <fstream>

std::ofstream binOut("archivo.bin", std::ios::binary); // Escritura
binaria
std::ifstream binIn("archivo.bin", std::ios::binary);   // Lectura
binaria

if (binOut.is_open() && binIn.is_open()) {
// Leer o escribir datos binarios
}
```

Ejemplo para guardar un objeto:

```
int dato = 123;
binOut.write(reinterpret_cast<const char*>(&dato), sizeof(dato));
```

Para leer:

```
int datoLeido;
binIn.read(reinterpret_cast<char*>(&datoLeido), sizeof(datoLeido));
```

Resumen de las opciones más comunes:

- `std::ios::in` — abrir para leer.
- `std::ios::out` — abrir para escribir.
- `std::ios::app` — abrir para agregar al final.
- `std::ios::trunc` — truncar (borrar contenido previo).
- `std::ios::binary` — modo binario (opcional, para archivos binarios).